Master Implementation Checklist: Hybrid Dual-Portal Sourcing Platform with joeyism/linkedin_scraper

This checklist guides the developer step-by-step through setting up, scripting, and testing the dual-pipeline recruiting workspace using direct browser scraping.

🗓️ Milestone 1: Configuration & Relational Store Setup

- [ ] **1.1 Workspace Layout:** Initialize git and confirm the file structure is correctly situated. Create placeholder files: `app.py`, `recruiting_db.json`.
- [ ] **1.2 Package Registration:** Populate `requirements.txt` with specific library versions:

```
streamlit>=1.35.0
openai>=1.30.0
plotly>=5.22.0
pypdf>=4.2.0
python-dotenv>=1.0.1
linkedin-scraper>=3.1.2
playwright>=1.40.0
pydantic>=2.0.0
```

- [ ] **1.3 Playwright Binary Installation:** Run installation routines locally to enable automated web crawling:

```
playwright install chromium
```

- [ ] **1.4 Environment Variables:** Create a `.env` file containing local API secrets:

```
OPENAI_API_KEY=your_openai_key_here
```

- [ ] **1.5 Flat-File Data Schema:** Design the JSON-backed database (`recruiting_db.json`) with relational tables:
  - [ ] `positions` : Dictionary indexing open roles with explicit keys: `{id, title, description, requirements, boolean_queries}` .
  - [ ] `candidates` : Relational state tracking indexed by candidate email: `{name, email, status, position_id, is_sourced, linkedin_url, profile_data, sourcing_pitch, outreach_email, match_results, custom_questions, answers, evaluation}` .

Milestone 2: 6-Agent Sourcing Engine Core

- [ ] **2.1 Resume Agent System Prompt:** Write the structured JSON parser instructions supporting the **Prestige Neutralizer** capability, converting specific schools/employers into generalized functional descriptions.
- [ ] **2.2 Employer Requirement Agent Prompt:** Implement the analyzer prompting loop to identify primary skill pillars and build searchable dynamic boolean queries.
- [ ] **2.3 LinkedIn Sourcing Agent Playwright Integration:** Build the asynchronous scraper module inside `app.py` :
  - [ ] Import `BrowserManager` and `PersonScraper` from `linkedin_scraper` .
  - [ ] Setup session persistence handlers. Design a setup panel to parse uploaded or pasted raw `session.json` strings.
  - [ ] Add automated error fallback logic to process simulated sandbox data profiles if Playwright crashes in restrictive headless environments.
- [ ] **2.4 Matching Agent Prompt (The Committee):** Author system instructions for the dual persona critical debate (Advocate vs Recruiter). Enforce structural score output metrics using:

$$\text{Composite Score} = \frac{\text{Technical} + \text{Domain} + \text{Culture}}{3}$$

Incorporate the **Trajectory Slope Score** formula in the prompt instructions to weigh learning speed, self-taught mechanics, and project momentum.

- [ ] **2.5 Candidate Interview Agent (Double-Phase Loop):**
  - [ ] Write Phase A (Generation) prompts to craft exactly 3 targeted technical questions targeting the gaps identified by the Matching Agent.
  - [ ] Write Phase B (Evaluation) prompts to grade string replies, outputting numerical marks (0-100) and actionable critiques.
- [ ] **2.6 Report Agent Prompt (Synthesizer):** Program instructions to compile the overall score outcomes to write highly persuasive, 2-sentence "Why This Person?" executive pitches and map out the personalized 3-week upskilling roadmap.

Milestone 3: Sourcing Console & Real Email Automation

- [ ] **3.1 Position Builder Forms:** Mount the text area element in Streamlit allowing managers to publish jobs. Connect input submissions to the **Employer Requirement Agent**.
- [ ] **3.2 Browser Scraper Interface:** Build the scraper console fields in `app.py`. Set up credentials input blocks for the LinkedIn session file or direct login profiles, alongside target invitation email fields.
- [ ] **3.3 Sourcing Pipeline Trigger:** Implement the `"Fetch, Scrape & Analyze Profile"` processor:
  - [ ] Trigger an asynchronous run looping through Playwright's page managers to fetch the user profile.
  - [ ] Pass the scraped JSON parameters directly to the **Matching Agent** and **Candidate Interview Agent**.
  - [ ] Render live step-by-step progress logging using `st.code()` mock screens.
- [ ] **3.4 Outreach SMTP System:** Configure an SMTP connection routine inside `app.py` utilizing standard Python components (`smtplib`, `email.mime`):
  - [ ] Setup sidebar controls to host authentication vectors: SMTP host, TLS port, sender login, and app password keys.
  - [ ] Trigger automated email dispatches automatically if match scores cross the manager's suitability threshold.

Milestone 4: Candidate Experience Portal & Progress Dashboard

- [ ] **4.1 Entrypoint Identity Routing Gate:** Configure the login prompt inside the Candidate Portal. Evaluate the entered string:
  - [ ] *Condition A (Email in Sourcing Database):* Check if `is_sourced == True` and state is `"Invited"`.
  - [ ] *Condition B (New Email Address):* Render the open application flow. Display target open roles, support PDF uploads, extract text using `pypdf.PdfReader`, and generate custom questions on the fly.

- [ ] **4.2 Sourced Candidate Welcome Board:**
  - [ ] Display high-visibility greetings displaying candidate achievements: *"Welcome back, [Candidate Name]! You have been hand-selected for our [Position Name] role based on your LinkedIn achievements."*
  - [ ] Render the personalized outreach email drafted by the **Report Agent** in an informational card to establish context.
- [ ] **4.3 Interactive Candidate Progress Tracker:**
  - [ ] Implement an interactive, stage-aware progress indicator using `st.progress()` to visually track the application workflow:
    - [ ] State `"Applied"` / `"Inbound Application Processed"`: Render progress bar at **33%**.
    - [ ] State `"Invited"` / `"Pre-Sourced LinkedIn Candidate"`: Render progress bar at **50%** (Sourcing verified).
    - [ ] State `"Screening Completed"`: Render progress bar at **100%**.
- [ ] **4.4 The Interactive Screening Canvas:** Build three sequential `st.text_area` modules inside the screening room. Populate labels using the generated custom questions.
- [ ] **4.5 Automated Feedback Display:**
  - [ ] If candidate status is `"Screening Completed"`, replace question prompts with a performance overview card using `st.info()`.
  - [ ] Display the automated grade score (0-100) alongside qualitative suggestions and the generated 3-week upskilling roadmap.

📅 Milestone 5: Hiring Manager Dashboard, Visual Analytics & Safety Guards

- [ ] **5.1 Executive KPI Metrics Strip:**
  - [ ] Render three horizontal metric cards at the top of the workspace:
    - [ ] *KPI 1:* **Active Open Positions** count.
    - [ ] *KPI 2:* **Total Sourced/Applied Candidates** count.
    - [ ] *KPI 3:* **Screening Interviews Completed** count.
- [ ] **5.2 Anonymized Bias Mitigation Interface:**
  - [ ] Place a prominent global `st.toggle("Activate Anonymized Blind Hiring Protocol")` at the top of the manager panel.

    - [ ] Write conditional formatting functions to replace candidate names and emails with randomized hashes or generic ID tokens (e.g., *Candidate #7291*) inside the dashboard views when enabled.
- [ ] **5.3 Sourced Shortlist Visualization:**
    - [ ] Construct a horizontal alignment chart using `plotly.graph_objects` comparing candidates.
    - [ ] Sort candidates dynamically based on their aggregated Matching Agent similarity scores.
- [ ] **5.4 Active Candidate Pipeline Board:**
    - [ ] Design a clean candidate summary list. Each card should feature:
        - [ ] Candidate Name/Hash, Target Role, Application Type (Inbound vs. Sourced).
        - [ ] Overall match fit percentage indicator.
        - [ ] The 1-sentence "Why This Person?" executive pitch generated by the **Report Agent**.
- [ ] **5.5 Analytical Detail Drill-down Panel:**
    - [ ] Map a dedicated expander (`st.expander()`) for each candidate to display deep-dive profiles:
        - [ ] Comparative debate panels splitting **Talent Advocate Pros** vs. **Critical Recruiter Cons** in side-by-side columns (`st.columns(2)`).
        - [ ] The tailored screening questions alongside the candidate's answers.
        - [ ] The automated evaluation grade and detailed 3-week on-boarding roadmap.

Milestone 6: Performance Optimization, Testing & Polish

- [ ] **6.1 Streamlit Cache Optimizations:** Wrap API query routines in `@st.cache_data` decorators to protect execution budgets during testing iterations.
- [ ] **6.2 Prompt Exception Sanitization:** Implement rigorous data validation handlers (`try/except` wrappers) around all `json.loads()` operations targeting agent returns to prevent UI crashes if an LLM outputs malformed text.
- [ ] **6.3 End-to-End Validation Testing:**
    - [ ] Run the open application pipeline from start to finish using a mock PDF resume. Verify the database updates correctly.